一、LLM开发入门：API调用

1.LLM界的“hello world”

1.1 准备工作

1.1.1 申请API_Key

地址：https://www.deepseek.com/



在官网的右上角点击“API开放平台”，进入API调用界面



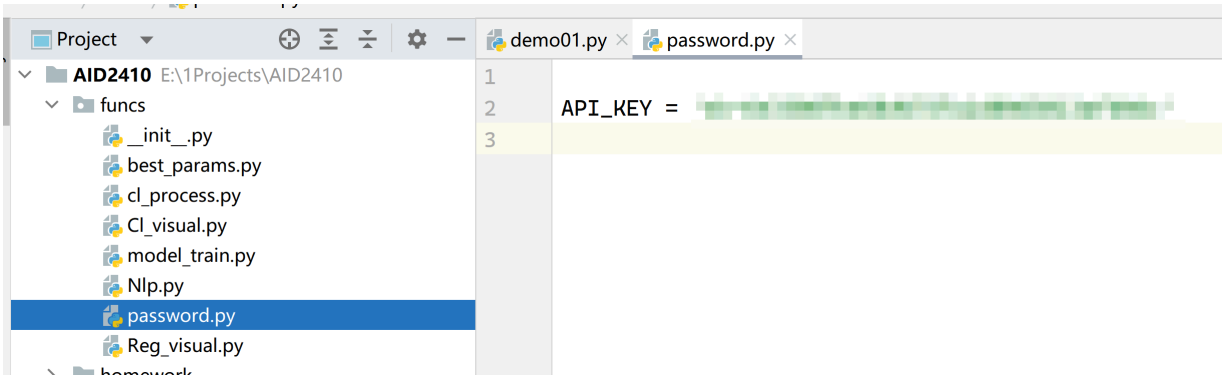
完成身份认证以及账号登录之后，点击API Keys可以进行API_Key的创建，点击下方的创建即可创建。



- Key是你能够调用API接口的关键，也是你的身份认真，请勿泄露
- 创建完成后会隐藏折叠起来，所以创建完成的时候，记得拷贝API_Key



- 点击复制后，会把API_Key存在剪贴板里面，但是并不安全，我们把它放在pycharm项目里面。



1.1.2 获取接口文档



打开接口文档，可以查看官方的接口调用说明。



这里点击python即可看到对应的调用说明。

curl python nodejs

```python
# Please install OpenAI SDK first: `pip3 install openai`

from openai import OpenAI

client = OpenAI(api_key="<DeepSeek API Key>", base_url="https://api.deepseek.com")

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Hello"},
    ],
    stream=False
)

print(response.choices[0].message.content)
```

1.1.3 准备环境

这里我们使用conda创建虚拟环境：

1.先打开anaconda prompt，创建 3.10 的python，起名叫`llm`

```
conda create -n llm python=3.10

# 创建完成之后
conda activate llm
```

2.安装必要的包

```
pip install openai
```

1.1.4 推理第一步

```python
from openai import OpenAI

API_KEY = 'your_api_key'
client = OpenAI(api_key=API_KEY, base_url="https://api.deepseek.com")

if __name__ == '__main__':
    response = client.chat.completions.create(
        model="deepseek-chat",
        messages=[
            {"role": "system", "content": "You are a helpful assistant"},
            {"role": "user", "content": "Hello"},
        ],
        stream=False
    )

    print(response.choices[0].message.content)
```

显示结果：

```
Hello! How can I assist you today? 😊
```

1.2 重新定义推理函数

1.2.1 调用讲解

```python
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Hello"},
    ],
    stream=False
)
```

1.model = 'deepseek-chat'

这里调用的是'deepseek'的v3版本，相对来说反应速度会更快一些。如果想让大模型进行深度思考，可以改为'deepseek-reasoner'

2.messages = ...

这里是向大模型传递的提示词，相对来说有些复杂，我们自定义的时候可以优化

- `system`消息：向模型传递指令（如设定角色、要求输出格式）
- `user`消息：用户当前输入内容

3.stream = False是否流式返回结果（False表示一次性返回完整响应）

1.2.2 自定义

```python
# password.py
from openai import OpenAI

API_KEY = 'your_api_key'

class Model:
    def __init__(self, model_name,base_url= "https://api.deepseek.com"):
        self.model_name = model_name
        self.base_url = base_url
        self.client = OpenAI(api_key=API_KEY, base_url=base_url)

    def invoke(self,prompt):
        response = self.client.chat.completions.create(
            model=self.model_name,
            messages=[{
                "role": "user",
                "content": prompt
            }],
            stream=False
        )
        return response.choices[0].message.content

llm = Model("deepseek-chat")
print(llm.invoke("你好"))
```

这样，我们自己构建了一个Model，并定义了一个invoke方法，就可以直接从funcs的password.py文件导入。

调用示例：

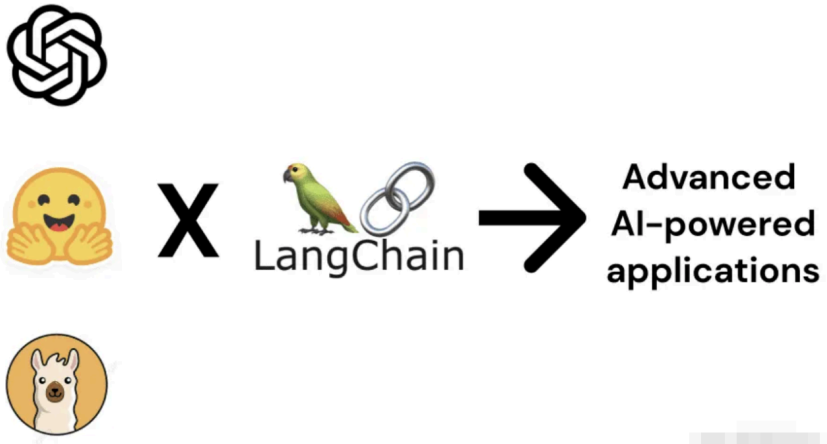
```python
from funcs.password import *

llm.invoke('hello world')
```

调用结果：

```
你好！😊 很高兴见到你～有什么我可以帮你的吗？
```

1.3 LangChain 开发库介绍



LangChain 是一个为大语言模型（LLM）和自然语言处理（NLP）应用程序提供开发支持的框架。它旨在简化与大模型相关的各种任务，例如文档查询、对话管理、信息提取、内容生成等。LangChain 提供了多种工具和组件，帮助开发者构建更高效、更可扩展的自然语言处理应用。

1.3.1 LangChain的火热发展

1.**架构设计与功能扩展**

LangChain 作为一个开源框架，提供了丰富的工具链，帮助开发者更轻松地构建以大语言模型为核心的应用。它封装了与外部数据源、API、文档存储和数据库的连接，并提供了对话流、任务管理、上下文管理等功能，极大地降低了使用大模型构建应用的复杂度。

LangChain 的设计目标是为开发者提供一个高效的、模块化的开发框架，让他们可以根据具体需求灵活组装应用。在 LangChain 的支持下，开发者可以快速实现数据读取、问答系统、对话管理、文件处理等应用场景。

2.**行业需求的推动**

随着大语言模型的普及，越来越多的行业和企业开始探索如何将这些强大的模型应用到实际场景中。LangChain 正是满足了这种需求，提供了一种标准化的方式来集成多个工具和技术，使得开发者可以专注于业务逻辑，而无需关心底层技术实现。特别是在知识库查询、自动化任务和文档处理等领域，LangChain 提供了非常强大的支持。

例如，开发者可以通过 LangChain 很容易地实现基于文本生成的自动问答系统、文章摘要系统、情感分析等，这些应用在客服、教育、医疗等行业中有着广泛的需求。

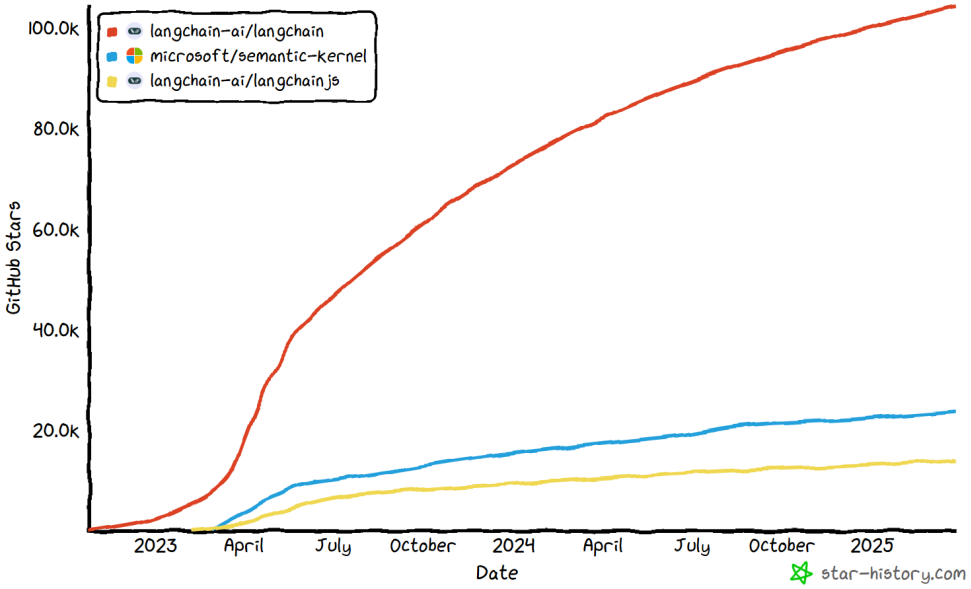
3.**开源和社区支持**

LangChain 作为一个开源项目，得到了广泛的社区支持。GitHub 上的活跃开发者和贡献者使得 LangChain 不断更新和完善。社区成员不仅提供了大量的工具和集成插件，还通过问题讨论、文档补充等方式，为新手开发者提供帮助。

此外，LangChain 还与多个知名的大模型平台（如 OpenAI、Hugging Face、百度千帆等）进行了深度集成，使得开发者可以轻松选择不同的模型进行调用，进一步提升了开发的灵活性和可扩展性。

1.3.2 LangChain 的挑战与前景

1.**学习曲线和复杂度**



尽管 LangChain 提供了强大的功能，但对初学者而言，掌握它的所有组件和功能可能需要一定的学习成本。LangChain 涉及到多个模块和外部集成，特别是在自定义功能、数据源接入和上下文管理等方面，开发者可能需要深入了解框架的工作原理。

为了降低学习门槛，LangChain 官方文档提供了详细的指南和教程，但对于没有深厚编程背景的开发者来说，可能需要较多的时间来熟悉相关概念和技术。

2.**计算资源需求**

LangChain 依赖于大模型和外部 API，这使得其在实际应用中可能会面临计算资源和成本的挑战。例如，调用大模型进行推理可能需要高性能的服务器和大量的计算资源，这对于小型企业或个人开发者而言，可能成为应用的障碍。

为了减少计算资源的消耗，LangChain 也支持一些优化技术，如模型压缩和推理加速，但如何平衡性能和计算资源的消耗仍然是一个挑战。

3.**模型和数据源的集成复杂性**

LangChain 提供了多种与外部模型和数据源的集成方式，但在复杂应用场景下，如何高效地管理多个数据源和模型的接口仍然需要谨慎设计。尤其是在不同模型和数据源之间进行切换时，可能会面临接口不兼容、数据格式不统一等问题。

为了简化集成过程，LangChain 提供了标准化的接口和预设的集成组件，但开发者仍然需要确保与目标数据源的匹配性，以及大模型的调用效率。

1.4 安装 LangChain 和相关依赖

1.4.1 Langchain 最新架构



自 LangChain 1.0 发布以来（2025年10月），整个框架被重新定位为四个互锁层：**模型与集成**、**LCEL组合语言**、**Agent与工具**，以及作为底层运行时的 **LangGraph**。

（1）**模型交互 Model I/O** — 依然存在，是最基础的模块

Prompt Template → Chat Model → Output Parser 这条链路没有变，但接口更统一，支持 1000+ 模型提供商。

（2） 检索增强 Retrieval / RAG — 依然是核心场景，没有大变化

文档加载 → 切块 → 向量化 → 检索，这条 RAG 流水线是 LangChain 的拿手好戏。

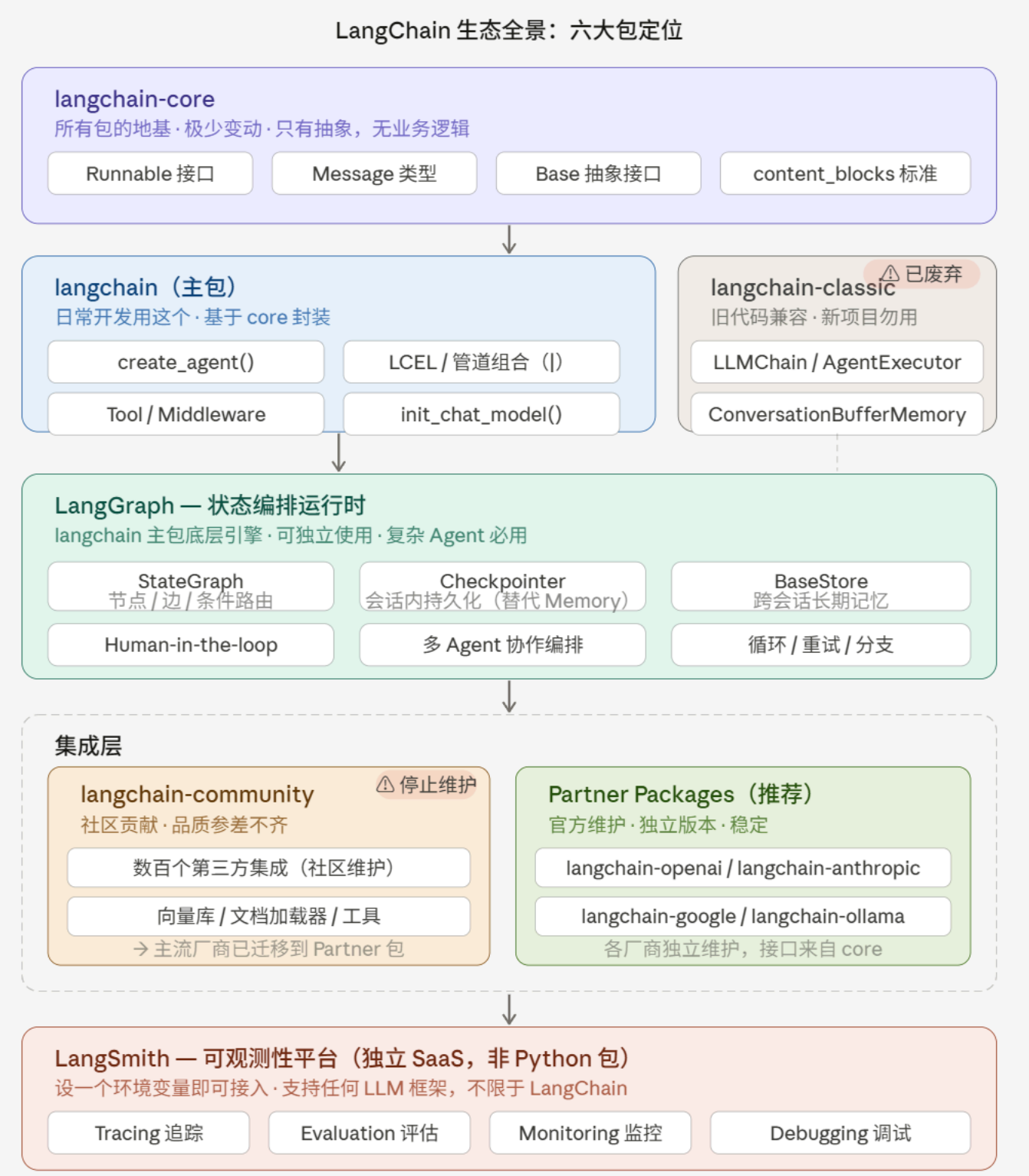
（3） 组合语言 LCEL — 这是取代旧版 Chain 的现代写法。

旧的 `LLMChain` 等被移入 `langchain-classic` 包维护，大量使用 Chain 的写法已成为过去式，现在通过 Runnable 接口统一组合。

(4) 智能体 Agent — 接口更简洁

从 `create_react_agent` 改名为 `create_agent`，底层由 LangGraph 运行时驱动，获得了持久化、可重放、Human-in-the-loop 等能力。

1.4.2 Langchain 生态全景



- **langchain-core** 是整个生态的地基。

所有提供商集成包使用的接口（Runnable、Message 类型、LLM 基类等）都定义在这里，它随 v1.0 同步升级，新增了 `content_blocks` 标准，让不同模型的输出结构统一。开发者一般不直接操作它，但它被所有包依赖。

- **langchain（主包）** 是日常开发的入口。

v1.0 精简了命名空间，聚焦于 Agent 构建所需的核心内容：`create_agent`、Messages、Tools、`init_chat_model` 等，大部分都是从 langchain-core 再导出的便捷入口。

- **langchain-classic** 是旧代码的"养老院"。

`LLMChain`、`AgentExecutor`、`ConversationBufferMemory`、`ConversationalRetrievalChain` 等所有废弃的类都搬到了这里，旧代码改一下 import 路径还能跑，但新项目不要碰。

- **LangGraph** 是运行时引擎，负责"状态"。

langchain 主包的 Agent 底层就跑在 LangGraph 上；简单场景用 langchain 的高层接口就够，复杂场景（多 Agent、循环、Human-in-the-loop）直接用 LangGraph 的 StateGraph 精细控制。

- **LangSmith** 不是 Python 包，是一个独立的云平台。

设置一个环境变量就能接入，提供追踪、评估、监控和调试能力，而且不限于 LangChain——任何 LLM 应用都可以用它。

截至编写课件时，所使用的langchain最新版为 1.2，为方便教学，建议大家统一安装该版本

```
# 终端 或者 anaconda prompt
pip install langchain==1.2 langchain-classic langchain-community langgraph
```

1.5 提示词介绍

1.5.1 什么是 Prompt

Prompt（提示词）是用户输入给大语言模型的一段文本信息，用于描述任务目标、提供上下文信息，并引导模型生成符合预期的输出结果。

对于大语言模型而言，Prompt 就像人与模型之间的沟通语言。用户通过 Prompt 告诉模型：

- 需要完成什么任务
- 应该扮演什么角色
- 输出什么内容
- 按照什么格式返回结果

例如：

```
请将下面的中文翻译成英文：

人工智能正在改变世界。
```

或者：

```
你是一名资深Python工程师，请解释什么是装饰器，并给出代码示例。
```

这些输入内容都属于 Prompt。

从本质上看，Prompt 就是一种利用自然语言向模型描述需求的方式。与传统软件开发通过编写程序控制逻辑不同，在大语言模型时代，我们更多是通过编写 Prompt 来引导模型完成任务。

因此，Prompt Engineering（提示词工程）的核心目标，就是设计出更清晰、更准确、更稳定的 Prompt，使模型能够输出更符合预期的结果。

1.5.2 Prompt 的基本结构

（1）Task（任务）

告诉模型需要完成什么工作。例如：

```
请将下面的中文翻译成英文。

请总结以下文章内容。
```

Task 是 Prompt 的核心，没有 Task 就没有 Prompt。

（2）Context（上下文）

为模型提供背景信息，例如：

```
以下内容来自公司内部技术文档：

.....
```

或者：

```
面向零基础学生进行讲解。
```

Context 的作用是缩小模型的理解范围。

（3）Role（角色）

为模型指定身份。例如：

```
你是一名资深Python工程师。
你是一名法律顾问。
```

Role 能够影响模型的表达方式和回答风格。

（4）Constraint（约束条件）

限制模型输出，例如：

```
不超过300字。
使用通俗易懂的语言。
不要输出代码。
```

约束条件可以提高输出稳定性。

（5）Output Format（输出格式）

规定结果格式，例如：

```
请以表格形式输出。
请以JSON格式输出。
请按照以下结构输出：

1.概念介绍
2.代码示例
3.总结
```

（6）总结

一个完整 Prompt 通常由以下部分组成：

```
Prompt = Task + Context + Role + Constraint + Output Format
```

对应的部分

```
Task        → 做什么
Context     → 在什么背景下做
Role        → 以什么身份做
Constraint  → 有什么限制
Format      → 如何输出
```

1.5.3 课堂练习：设计一本原创小说

（1）**任务目标**

练习把 Prompt 的五个基本部分（Role、Task、Context、Constraint、Output Format）完整、清晰地写出来，并观察不同写法对模型输出质量的影响。

（2）**具体要求**

- 从以下方向任选一个（或自创）：

  修仙、科幻、末日生存、都市异能、校园青春、悬疑推理、无限流、穿越、历史架空、游戏竞技。
- **自己编写一个 Prompt**，必须包含以下五个部分（建议用小标题分开写）：
  - **Role（角色）**
  - **Task（任务）**
  - **Context（上下文）**
  - **Constraint（约束条件）**
  - **Output Format（输出格式）**
- 把写好的 Prompt 交给大模型，生成结果后思考以下内容：
  - 你觉得 Prompt 里哪个部分写得比较好？为什么？
  - 生成结果有哪些地方让你满意或不满意？
  - 如果重写，你最想修改 Prompt 的哪一部分？

（3）**模型输出内容要求**（在你的 Output Format 里写清楚）

模型需要输出以下内容：

- 小说名称
- 故事背景
- 主角设定
- 核心冲突
- 主要角色
- 前三章剧情规划

（4）**注意**

- 重点是练习 Prompt 的结构和写法，不是让模型写完整小说。
- 生成内容要有一定创意和逻辑性即可。
- 长度适中（模型输出控制在 1500 字左右比较合适）。

1.5.4 设计对话小游戏

练习名称：哄对象开心（对话状态追踪游戏）

学习目标：通过本次练习，你将：

- 练习为**互动对话 + 状态维护**类任务编写 Prompt。
- 重点掌握如何用 **Constraint** 控制模型严格遵守格式、维护内部状态（原谅值）、保持角色一致性。
- 体会 Output Format 在实际应用中的重要性（便于后续解析或开发简单界面）。
- 对比上一个练习（小说概念生成），理解不同场景下 Prompt 设计的差异。

任务描述：请你自己编写一个 Prompt，让大模型运行下面这个互动对话小游戏。

游戏规则（写在你的 Prompt Context 或 Constraint 里）：

- 游戏开始时，用户可能会提供"对象生气的原因"，如果没有提供则随机生成一个合理的原因。
- 初始原谅值 = 20/100。
- 玩家每回复一句，模型根据内容给出 -10、-5、0、+5、+10 的分数，并更新原谅值。
- 模型要以"对象"的身份回复，语气要符合当前心情（难哄、敏感）。
- 游戏进行中必须严格使用固定格式输出。
- 当原谅值 ≥ 100 → 游戏胜利；当原谅值 ≤ 0 → 游戏失败。
- 游戏结束后，根据完整对话生成：
    - 一首简短的诗（总结这次互动）
    - 一段图像生成提示词（胜利用温暖喜庆风格，失败用悲伤凄凉风格）

输出格式要求（必须在 Prompt 中严格规定）：

游戏进行中：

```
(心情)对象说的话
得分：+5 / -10 等
原谅值：当前值/100
```

游戏开始时：

```
游戏开始，请现在开始哄你的对象开心吧！
得分：0
原谅值：20/100
```

游戏结束时（胜利或失败）：在最后一条回复后加上：

- 胜利：`恭喜你通关了！`
- 失败：`游戏结束，你被甩了！` + 生气原因总结

- 然后统一加上：

```
【结局诗】
（4-6 行诗）

【图片生成提示】
（详细的图像描述）
```

简短思考：

- Prompt 里哪个部分最关键？为什么？
- 模型有没有出现格式错误或角色崩坏的情况？
- 如果重写，你会加强哪一部分的 Constraint？